Employee Management CRUD Application Report

Overview

This project is a full-stack Employee Management CRUD application built with React for the frontend, Spring Boot for the backend, and MySQL for the database. The application supports Create, Read, Update, and Delete operations through a React user interface that sends HTTP requests to Spring Boot REST APIs, which then persist and retrieve employee data from MySQL.[cite:155] [cite:188] [cite:112]

Objective

The objective of this project is to implement a simple and complete CRUD workflow for employee records. The application demonstrates frontend form handling, API integration with Axios, layered backend development using controller and repository patterns, and database persistence using Spring Data JPA with MySQL.[cite:155] [cite:188]

Tools and Technologies

Component	Technology	Purpose
Frontend	React	Build the user interface and manage client-side state.[cite:155]
Backend	Spring Boot	Build REST APIs and handle business/data flow.[cite:188]
Database	MySQL	Store employee records persistently.[cite:112]
API Client	Axios	Send HTTP requests from React to backend APIs.[cite:155]
ORM	Spring Data JPA	Map Java objects to database tables and simplify CRUD operations.[cite:188]

Project Structure

The project is split into separate backend and frontend modules so both sides can be developed and run independently. The frontend contains the React components, service layer, and styling, while the backend contains the Spring Boot application, entity, repository, controller, and configuration files.

```
employee-management/
├── backend/
│   └── src/main/
│       ├── java/com/example/employeemanagement/
│       │   ├── EmployeeManagementApplication.java
│       │   ├── controller/EmployeeController.java
│       │   ├── model/Employee.java
│       │   └── repository/EmployeeRepository.java
│       └── resources/application.properties
```

```
└── frontend/
    └── src/
        ├── App.js
        ├── App.css
        ├── components/
        │   ├── EmployeeForm.js
        │   └── EmployeeList.js
        └── services/
            └── EmployeeService.js
```

Database Setup

Open MySQL Workbench and create the project database before starting the backend. MySQL supports creating the database with a simple `CREATE DATABASE employee_db;` command. [cite:112]

```
CREATE DATABASE employee_db;
SHOW DATABASES;
```

Backend Implementation

The Spring Boot backend reads datasource and server configuration from `application.properties`, and the `server.port` property can be used to move the application away from the default 8080 port when needed.[cite:188] [cite:186]

application.properties

```
spring.application.name=employee-management

server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/employee_db?useSSL=false&serverTi
spring.datasource.username=root
spring.datasource.password=YOUR_MYSQL_PASSWORD
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

Main Application Class

```
package com.example.employeemanagement;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class EmployeeManagementApplication {
```

```
    public static void main(String[] args) {
        SpringApplication.run(EmployeeManagementApplication.class, args);
    }
}
```

Employee Entity

```
package com.example.employeemanagement.model;

import jakarta.persistence.*;

@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name", nullable = false)
    private String firstName;

    @Column(name = "last_name", nullable = false)
    private String lastName;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String department;

    @Column(nullable = false)
    private Double salary;

    public Employee() {
    }

    public Employee(Long id, String firstName, String lastName, String email, Stri
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.department = department;
        this.salary = salary;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getFirstName() {
```

```
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public Double getSalary() {
        return salary;
    }

    public void setSalary(Double salary) {
        this.salary = salary;
    }
}
```

Employee Repository

```
package com.example.employeemanagement.repository;

import com.example.employeemanagement.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
}
```

Employee Controller

```
package com.example.employeemanagement.controller;

import com.example.employeemanagement.model.Employee;
import com.example.employeemanagement.repository.EmployeeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/employees")
@CrossOrigin(origins = "http://localhost:3000")
public class EmployeeController {

    @Autowired
    private EmployeeRepository employeeRepository;

    @GetMapping
    public List<Employee> getAllEmployees() {
        return employeeRepository.findAll();
    }

    @GetMapping("/{id}")
    public Employee getEmployeeById(@PathVariable Long id) {
        Optional<Employee> employee = employeeRepository.findById(id);
        return employee.orElse(null);
    }

    @PostMapping
    public Employee createEmployee(@RequestBody Employee employee) {
        return employeeRepository.save(employee);
    }

    @PutMapping("/{id}")
    public Employee updateEmployee(@PathVariable Long id, @RequestBody Employee emp
        Employee employee = employeeRepository.findById(id).orElse(null);

        if (employee != null) {
            employee.setFirstName(employeeDetails.getFirstName());
            employee.setLastName(employeeDetails.getLastName());
            employee.setEmail(employeeDetails.getEmail());
            employee.setDepartment(employeeDetails.getDepartment());
            employee.setSalary(employeeDetails.getSalary());
            return employeeRepository.save(employee);
        }

        return null;
    }

    @DeleteMapping("/{id}")
    public String deleteEmployee(@PathVariable Long id) {
        employeeRepository.deleteById(id);
        return "Employee deleted successfully";
```

```
    }
}
```

Frontend Implementation

Create React App supports stylesheet imports such as `import "./App.css";`, which allows component styling to be kept in a dedicated CSS file and applied through JSX `className` attributes.[cite:155]

EmployeeService.js

```
import axios from "axios";

const BASE_URL = "http://localhost:8081/api/employees";

class EmployeeService {
  getAllEmployees() {
    return axios.get(BASE_URL);
  }

  addEmployee(employee) {
    return axios.post(BASE_URL, employee);
  }

  updateEmployee(id, employee) {
    return axios.put(`${BASE_URL}/${id}`, employee);
  }

  deleteEmployee(id) {
    return axios.delete(`${BASE_URL}/${id}`);
  }
}

export default new EmployeeService();
```

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

App.js

```
import React, { useEffect, useState } from "react";
import "./App.css";

import EmployeeForm from "./components/EmployeeForm";
import EmployeeList from "./components/EmployeeList";
import EmployeeService from "./services/EmployeeService";

function App() {
  const [employees, setEmployees] = useState([]);
  const [selectedEmployee, setSelectedEmployee] = useState(null);

  const fetchEmployees = async () => {
    try {
      const response = await EmployeeService.getAllEmployees();
      setEmployees(response.data);
    } catch (error) {
      console.error("Error fetching employees:", error);
      alert("Error fetching employees");
    }
  };

  useEffect(() => {
    fetchEmployees();
  }, []);

  const handleEdit = (employee) => {
    setSelectedEmployee(employee);
  };

  const clearSelectedEmployee = () => {
    setSelectedEmployee(null);
  };

  return (
    <div className="app-container">
      <div className="header">
        <h1>Employee Management System</h1>
      </div>

      <div className="content">
        <div className="section">
          <h2 className="section-title">
            {selectedEmployee ? "Edit Employee" : "Add Employee"}
          </h2>
          <EmployeeForm
            selectedEmployee={selectedEmployee}
            refreshEmployees={fetchEmployees}
            clearSelectedEmployee={clearSelectedEmployee}
          />
        </div>

        <div className="section">
          <h2 className="section-title">Employee List</h2>
          <div className="table-container">
```

```
            <EmployeeList
              employees={employees}
              onEdit={handleEdit}
              refreshEmployees={fetchEmployees}
            />
          </div>
        </div>
      </div>
    </div>
  );
}

export default App;
```

EmployeeForm.js

```
import React, { useEffect, useState } from "react";
import EmployeeService from "../services/EmployeeService";

function EmployeeForm({ selectedEmployee, refreshEmployees, clearSelectedEmployee
  const [employee, setEmployee] = useState({
    firstName: "",
    lastName: "",
    email: "",
    department: "",
    salary: ""
  });

  useEffect(() => {
    if (selectedEmployee) {
      setEmployee(selectedEmployee);
    } else {
      resetForm();
    }
  }, [selectedEmployee]);

  const resetForm = () => {
    setEmployee({
      firstName: "",
      lastName: "",
      email: "",
      department: "",
      salary: ""
    });
  };

  const handleChange = (e) => {
    const { name, value } = e.target;
    setEmployee({ ...employee, [name]: value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
```

```
      if (selectedEmployee && selectedEmployee.id) {
        await EmployeeService.updateEmployee(selectedEmployee.id, employee);
        alert("Employee updated successfully");
      } else {
        await EmployeeService.addEmployee(employee);
        alert("Employee added successfully");
      }

      resetForm();
      clearSelectedEmployee();
      refreshEmployees();
    } catch (error) {
      console.error("Error saving employee:", error);
      alert("Error saving employee");
    }
  };

  return (
    <div>
      <form onSubmit={handleSubmit}>
        <div className="form-group">
          <label>First Name:</label>
          <input
            type="text"
            name="firstName"
            value={employee.firstName}
            onChange={handleChange}
            required
          />
        </div>

        <div className="form-group">
          <label>Last Name:</label>
          <input
            type="text"
            name="lastName"
            value={employee.lastName}
            onChange={handleChange}
            required
          />
        </div>

        <div className="form-group">
          <label>Email:</label>
          <input
            type="email"
            name="email"
            value={employee.email}
            onChange={handleChange}
            required
          />
        </div>

        <div className="form-group">
          <label>Department:</label>
          <input
```

```
            type="text"
            name="department"
            value={employee.department}
            onChange={handleChange}
            required
          />
        </div>

        <div className="form-group">
          <label>Salary:</label>
          <input
            type="number"
            name="salary"
            value={employee.salary}
            onChange={handleChange}
            required
          />
        </div>

        <div className="form-buttons">
          <button type="submit" className="btn btn-primary">
            {selectedEmployee ? "Update Employee" : "Add Employee"}
          </button>

          <button
            type="button"
            className="btn btn-secondary"
            onClick={() => {
              resetForm();
              clearSelectedEmployee();
            }}
          >
            Clear
          </button>
        </div>
      </form>
    </div>
  );
}

export default EmployeeForm;
```

EmployeeList.js

```
import React from "react";
import EmployeeService from "../services/EmployeeService";

function EmployeeList({ employees, onEdit, refreshEmployees }) {
  const handleDelete = async (id) => {
    const confirmDelete = window.confirm("Are you sure you want to delete this emplo
    if (!confirmDelete) {
      return;
    }

    try {
```

```
      await EmployeeService.deleteEmployee(id);
      alert("Employee deleted successfully");
      refreshEmployees();
    } catch (error) {
      console.error("Error deleting employee:", error);
      alert("Error deleting employee");
    }
  };

  return (
    <div>
      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>First Name</th>
            <th>Last Name</th>
            <th>Email</th>
            <th>Department</th>
            <th>Salary</th>
            <th>Actions</th>
          </tr>
        </thead>

        <tbody>
          {employees.length > 0 ? (
            employees.map((employee) => (
              <tr key={employee.id}>
                <td>{employee.id}</td>
                <td>{employee.firstName}</td>
                <td>{employee.lastName}</td>
                <td>{employee.email}</td>
                <td>{employee.department}</td>
                <td>{employee.salary}</td>
                <td>
                  <div className="action-buttons">
                    <button
                      className="btn btn-primary"
                      onClick={() => onEdit(employee)}
                    >
                      Edit
                    </button>

                    <button
                      className="btn btn-danger"
                      onClick={() => handleDelete(employee.id)}
                    >
                      Delete
                    </button>
                  </div>
                </td>
              </tr>
            ))
          ) : (
            <tr>
              <td colSpan="7" className="no-data">
```

```
                No employees found
              </td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  );
}

export default EmployeeList;
```

App.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(135deg, #e0e7ff, #f5f3ff);
  min-height: 100vh;
  padding: 20px;
}

.app-container {
  max-width: 1200px;
  margin: 0 auto;
  background: #ffffff;
  border-radius: 18px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.08);
  overflow: hidden;
}

.header {
  background: linear-gradient(135deg, #4f46e5, #7c3aed);
  color: white;
  text-align: center;
  padding: 30px 20px;
}

.header h1 {
  font-size: 2.2rem;
  font-weight: 700;
}

.content {
  padding: 30px;
}

.section {
  margin-bottom: 35px;
}
```

```
.section-title {
  font-size: 1.6rem;
  margin-bottom: 20px;
  color: #1f2937;
}

.form-group {
  margin-bottom: 16px;
}

.form-group label {
  display: block;
  margin-bottom: 6px;
  font-weight: 600;
  color: #374151;
}

.form-group input {
  width: 100%;
  max-width: 420px;
  padding: 10px 14px;
  border: 1px solid #d1d5db;
  border-radius: 10px;
  font-size: 1rem;
  transition: 0.3s ease;
}

.form-group input:focus {
  outline: none;
  border-color: #4f46e5;
  box-shadow: 0 0 0 3px rgba(79, 70, 229, 0.15);
}

.form-buttons {
  display: flex;
  gap: 12px;
  margin-top: 18px;
  flex-wrap: wrap;
}

.btn {
  border: none;
  padding: 10px 18px;
  border-radius: 10px;
  cursor: pointer;
  font-size: 0.95rem;
  font-weight: 600;
  transition: 0.3s ease;
}

.btn-primary {
  background: #4f46e5;
  color: white;
}
```

```
.btn-primary:hover {
  background: #4338ca;
}

.btn-secondary {
  background: #e5e7eb;
  color: #111827;
}

.btn-secondary:hover {
  background: #d1d5db;
}

.btn-danger {
  background: #dc2626;
  color: white;
}

.btn-danger:hover {
  background: #b91c1c;
}

.action-buttons {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}

.table-container {
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
  background: white;
}

thead {
  background: #f3f4f6;
}

th,
td {
  padding: 14px 12px;
  text-align: left;
  border-bottom: 1px solid #e5e7eb;
}

th {
  color: #111827;
  font-weight: 700;
}

tr:hover {
  background: #f9fafb;
```

```
}

.no-data {
  text-align: center;
  color: #6b7280;
  padding: 20px;
  font-style: italic;
}

@media (max-width: 768px) {
  .content {
    padding: 20px;
  }

  .header h1 {
    font-size: 1.7rem;
  }

  .section-title {
    font-size: 1.3rem;
  }

  .form-group input {
    max-width: 100%;
  }

  th,
  td {
    font-size: 0.9rem;
    padding: 10px 8px;
  }
}
```

Execution Steps

Follow the steps in order so that the backend, database, and frontend connect correctly.

1. Create the `employee_db` database in MySQL.[cite:112]
2. Configure `application.properties` with the correct MySQL username, password, and `server.port=8081`.[cite:188] [cite:186]
3. Run the Spring Boot main class from Eclipse so the REST API starts on `http://localhost:8081`.[cite:188]
4. Create the React app files and install Axios using `npm install axios`.[cite:155]
5. Start the React app using `npm start`, which typically serves the app on `http://localhost:3000` in a Create React App setup.[cite:155]
6. Open the app in the browser and perform add, edit, delete, and list operations.

Expected Result

When the project runs successfully, the UI shows a styled Employee Management System page with an Add/Edit Employee form and an employee table. The React app sends requests through Axios to Spring Boot APIs, and those APIs read and write employee records in MySQL to complete the CRUD cycle.[cite:155] [cite:188] [cite:112]

Issue Resolved During Development

A duplicate “Add Employee” heading appeared because the title was rendered in both `App.js` and `EmployeeForm.js`. The corrected version keeps the conditional heading only in `App.js` and removes the extra `<h2>` from `EmployeeForm.js`, which ensures the interface shows a single heading that changes between Add Employee and Edit Employee based on selection.

Conclusion

This project demonstrates a complete full-stack CRUD application using React, Spring Boot, and MySQL. It is suitable for academic submission, interview demonstration, and practical learning because it covers frontend forms, API calls, backend REST endpoints, and persistent database operations in one integrated project.[cite:155] [cite:188] [cite:112]